RAPPEL · JOUR 3

Jour 4 : interface et API protégée

Rappel Postman · 10 min
Cours · 15 min
TD guidé · 65 min

Notre application « Mes documents »

Rappel : une API protégée

La connexion vérifie l’e-mail et le mot de passe.
Sanctum renvoie un token personnel.
Le client présente ce token pour les routes protégées.

Authentification : qui fait la demande ?

Rappel : chaque document a un propriétaire

Laravel cherche les documents de la personne connectée.
Un document appartenant à une autre personne répond 404.
Les anciens documents sans propriétaire ne figurent dans aucune liste.

Autorisation : cette personne a-t-elle accès ?

Postman : vérifier la connexion

1. GET /api/documents sans token : 401.
2. POST /api/login avec e-mail et mot de passe : 200 et token.
3. Authorization > Bearer Token, puis GET /api/documents : 200.

En-tête : Accept: application/json

Postman : vérifier le fichier

Amina crée un document et note son identifiant.
Elle envoie un PDF avec Body > form-data, clé file de type File.
Amina télécharge le PDF ; Karim reçoit 404 pour ce document.

Utiliser les comptes et fichiers fictifs de formation

Pourquoi adapter l’interface ?

L’interface du jour 2 sait appeler l’API avec Fetch.
L’API du jour 3 demande maintenant un token.
Sans adaptation, les demandes de documents reçoivent 401.

Objectif : relier les deux étapes du projet

Connexion depuis JavaScript

Le formulaire envoie un POST JSON vers /api/login.
En cas de succès, la réponse contient le token Sanctum.
Le token reste en mémoire dans la page pendant le TD.

Le rechargement demande une nouvelle connexion

Appeler une route protégée

Fetch ajoute Authorization: Bearer suivi du token.
Laravel identifie l'utilisateur avant de chercher ses documents.
La liste affichée dépend du compte connecté.

Un token est un secret : ne pas le publier ni le partager

Authentification : du TD à une application web

Notre TD : token Sanctum en mémoire dans la page.

Après rechargement, l'utilisateur se reconnecte.

Une application web peut utiliser une session et un cookie.
La configuration doit aussi prévoir la protection CSRF.

Le token de notre TD n'est pas un JWT.

Deux requêtes pour un PDF

1. POST /api/documents crée le titre et la catégorie.
2. POST /api/documents/{id}/file envoie le PDF.

Si l’envoi échoue, le document existe déjà.

Le corps du fichier utilise FormData

Envoyer un fichier avec FormData

La clé attendue par Laravel est file.
Le navigateur construit le contenu multipart.
Laravel vérifie la taille, le type et la signature du PDF.

Ne pas définir soi-même le Content-Type du FormData

Télécharger un fichier privé

Un lien direct ne peut pas ajouter notre en-tête Bearer.
Fetch demande le fichier avec le token.
JavaScript transforme la réponse en Blob et lance le téléchargement.

Laravel contrôle le propriétaire avant de répondre

Comprendre les réponses

401 : connexion absente, expirée ou révoquée.
404 : document ou fichier non accessible.
422 : données ou fichier refusés. 409 : PDF déjà présent.

Afficher un message utile et conserver les données saisies

Déconnexion et vérification

La déconnexion demande la révocation du token.
La page efface ensuite les données affichées.
Changer de compte permet de vérifier la séparation des listes.

TD : construire puis vérifier chaque étape

COURS · 25 MIN

Découvrir Valkey

Une mémoire rapide pour retrouver des données déjà demandées.

PostgreSQL reste la source de référence de notre exercice.



Observer le rôle de chaque composant

Valkey : plusieurs utilisations

Cache : garder une copie temporaire des données.

Files d’attente : conserver les tâches à traiter.

Rate Limiting : compter les requêtes pendant un délai.

Sessions : retrouver la session d’un utilisateur.

Dans notre TD : cache et compteurs de requêtes.

D’autres usages de Valkey

Verrous : coordonner des traitements concurrents.

Pub/Sub et streams : transmettre des événements.

Compteurs et classements : suivre et classer des valeurs.

Géolocalisation : trouver des lieux proches.

Ces usages sont présentés ici comme des exemples.

File d’attente : qui fait le travail ?

Exemple : Amina envoie un PDF.

Laravel place une tâche d’analyse dans une file.

Un worker Laravel récupère et exécute cette tâche.

Valkey peut conserver la file ; sa persistance se configure.

Une queue, Pub/Sub et un stream ont des rôles distincts.

Quel problème voulons-nous résoudre ?

Amina actualise sa liste plusieurs fois.
Sans cache, Laravel relit PostgreSQL pour chaque demande.

Comment éviter ces lectures répétées ?



Une copie temporaire peut servir aux prochaines lectures

Une clé et une valeur

Clé : documents:user:1
Valeur : [Contrat, Facture]

Exemple : la liste du compte 1
reste en cache pendant cinq minutes.

La clé doit identifier le bon utilisateur

Mémoire et persistance

Valkey travaille principalement en mémoire.
Il peut aussi conserver des données sur disque.
Dans ce TD, la persistance est désactivée.
PostgreSQL reste la source de référence.

Les PDF restent dans le stockage privé Laravel

Les services de notre projet

PostgreSQL : les documents de référence.
Valkey : une copie temporaire en mémoire.
Laravel : décide où lire la liste.
Docker : démarre les services.
pgAdmin : permet de voir la base.



Docker Compose organise leur démarrage

Communiquer avec Valkey

Valkey est issu de Redis OSS 7.2.
Predis est le client PHP utilisé dans ce TD.
Laravel emploie son pilote « redis »
pour joindre le service nommé valkey.

Valkey est compatible avec le protocole Redis

Une configuration locale

```
CACHE_STORE=redis
REDIS_CLIENT=predis
REDIS_HOST=valkey
```

Le fichier .env existant conserve sa clé et ses autres valeurs

Vérifier que Valkey répond

docker compose exec valkey valkey-cli ping

Réponse attendue : PONG

Un PONG vérifie le serveur, pas encore la connexion Laravel

TD · 30 MIN

À vous : connexion à Valkey

Dans le guide : partie 1, étapes 1 à 4.
Ajouter le service Docker et le client PHP.
Configurer Laravel, puis écrire et relire une clé.

Pause : quel composant conserve la valeur ?

Le cache avec Laravel

Le serveur peut garder une copie de la réponse déjà construite.

Une demande suivante peut réutiliser cette copie tant qu’elle reste valide.



Illustration : la petite étagère représente le cache

Première lecture : cache absent

1. Amina demande sa liste.
2. La clé manque dans Valkey.
3. Laravel lit PostgreSQL et garde une copie pendant cinq minutes.
4. Laravel renvoie la liste.



Cache vide : Laravel charge les documents depuis PostgreSQL

Lecture suivante : cache présent

1. Amina redemande sa liste.
2. Laravel cherche la même clé.
3. Valkey fournit la copie valide.
4. Laravel renvoie cette liste.



Cache présent : la liste peut être réutilisée

Cache::remember

```
Cache::remember(
  $key, now()->addMinutes(5),
  fn () => $request->user()->documents()->get()->toArray()
);
```

Absent : calculer et stocker. Présent : réutiliser.

Une liste par utilisateur

Amina : documents:user:1
Karim : documents:user:2

Chaque clé identifie un utilisateur.
Les listes restent séparées.



Le cache ne remplace pas le contrôle d’accès Laravel

Durée de vie : le TTL

La clé expire après cinq minutes.
La prochaine demande relit PostgreSQL.
Le TTL indique le temps restant.

Une modification invalide aussi la copie.

Une durée courte limite l’ancienneté possible

Observer les clés

Dans le conteneur Valkey :
valkey-cli -n 1 --scan --pattern *documents:user:*

Le préfixe et l’identifiant distinguent la clé.

La base logique 1 correspond au cache du projet

TD · 40 MIN

À vous : mettre la liste en cache

Dans le guide : partie 2, étapes 5 à 7.
Créer un document fictif et adapter index.
Observer la clé, son TTL et les deux utilisateurs.

Pause : qu’arrive-t-il quand la clé expire ?

Pourquoi invalider ?

Une écriture change PostgreSQL.
La copie en mémoire peut rester ancienne.

On retire alors cette copie avant
la prochaine lecture.



Illustration : retirer une copie devenue ancienne

La liste d’Amina a changé

Au départ, la liste contient 3 documents.
Amina ajoute une facture : PostgreSQL en a 4.
L’ancienne copie en cache en contient encore 3.

Sans invalidation, la liste peut être périmée.

Un résultat rapide peut être périmé

La réponse après une modification

1. Laravel enregistre le changement dans PostgreSQL.
2. Laravel supprime la copie du compte concerné.
3. Le prochain GET lit la liste à jour et la remet en cache.

Cache::forget retire une seule clé

Quelles actions invalident ?

Créer, modifier ou supprimer un document.
Ajouter un PDF modifie aussi le document.
Une action réussie invalide la liste du propriétaire.

La liste des autres utilisateurs reste en cache

TTL et invalidation

TTL : la copie expire après un délai.
Invalidation : une action retire la copie devenue ancienne.
Les deux mécanismes sont complémentaires.

La cohérence en cas de concurrence demande une étude adaptée

TD · 30 MIN

À vous : actualiser la liste

Dans le guide : partie 3, étapes 8 et 9.
Ajouter Cache::forget après les écritures.
Observer la clé avant et après PUT, DELETE et GET.

Pause : pourquoi supprimer après l'écriture réussie ?

Ce que nous avons vérifié

Valkey répond et Laravel peut lui parler.
Chaque utilisateur possède sa propre clé.
La liste se reconstruit après une invalidation.

La suite abordera la limitation des requêtes

Limiter les requêtes d’une API

Un compteur suit les demandes d’un utilisateur pendant un délai.

Au-delà de la limite, Laravel lui demande de patienter.



L’illustration représente un contrôle temporaire

Pourquoi limiter les demandes ?

Éviter qu’un client multiplie les demandes trop vite.
Partager les ressources entre les utilisateurs.
Réduire certains abus et protéger l’application.

Une limite ne remplace pas les autres protections

Trois éléments d’une limite

Action : GET /api/documents.
Maximum : cinq demandes.
Fenêtre : une minute pour chaque compte.

La sixième demande reçoit une réponse 429.

La valeur cinq sert à observer le résultat en classe

Les réponses à observer

Demandes autorisées : 200 OK.
Limite atteinte : 429 Too Many Requests.
Retry-After indique le délai avant un nouvel essai.

Le client attend avant de renvoyer sa demande

Un compteur pour chaque utilisateur

Amina et Karim ont chacun leur compteur.
Un nouveau token pour Amina garde le même quota.
Laravel identifie la personne avant de compter.

Le groupe auth:sanctum protège toujours la route

Deux limites distinctes

Connexion : POST /api/login, 10 par minute.
Liste : GET /api/documents, 5 par minute et par utilisateur.
Les autres routes ne sont pas concernées par la nouvelle limite.

La limite du TD porte seulement sur la liste

Où est conservé le compteur ?

Laravel utilise le cache configuré pour la limitation.
Dans notre TD, CACHE_STORE=redis utilise Valkey.
Le compteur et la liste en cache ont des rôles différents.

Le compteur sert à appliquer un quota temporaire

Définir puis appliquer

RateLimiter::for définit un limiteur nommé.
Limit::perMinute(5)->by(...) choisit le quota.
La route utilise throttle:documents.

L’authentification précède le limiteur de liste

Que verra-t-on dans Postman ?

1. Amina envoie cinq demandes : 200.
2. La sixième demande reçoit 429.
3. Karim peut utiliser son propre quota.
4. Après le délai, Amina peut réessayer.

Un client déjà actif peut avoir utilisé une partie du quota

TD · 25 MIN

À vous : tester la limitation

Dans le guide Valkey, partie 4 : étapes 10 à 15.
Définir le compteur et modifier la route de liste.
Observer 200, 429 et Retry-After avec Postman.

Pause : pourquoi les comptes ont-ils deux compteurs ?

BILAN · 10 MIN

Vérifications du jour 4

Liste protégée : 401 sans token, 200 avec token valide.
Documents : chaque compte voit sa propre liste.
PDF : seul son propriétaire peut le télécharger.

Si nécessaire, attendre la fin du quota avant le GET

BILAN · 10 MIN

Cache et limitation

Valkey répond PONG et Laravel peut le joindre.
Une clé de liste existe par utilisateur.
Une écriture invalide la liste concernée.
Le dépassement du quota renvoie 429.

Ne pas vider tout le cache pour faire le test

BILAN · 10 MIN

Questions pour les étudiants

Quel est le rôle de Laravel, PostgreSQL et Valkey ?
Quelle différence entre authentifier et autoriser ?
Pourquoi invalider une copie en cache ?
Que signifie la réponse 429 ?

Expliquer les réponses avec le projet de documents

Les 300 minutes de la journée

Interface et API protégée : 90 minutes.
Valkey, cache et invalidation : 160 minutes.
Rate limiting, TD et bilan : 50 minutes.

Prochaine étape envisagée : les tests de charge avec k6